LLM Developer Intern – Technical Assessment

Mode: Individual
Duration: 3–5 Days
Stack Options: Python (FastAPI) or JavaScript (Node.js + Express)
Goal: Evaluate your ability to build backend services that integrate LLM APIs, Vector Databases, and RAG logic.

Test Case 1 – LLM API Wrapper

Objective: Test backend API integration and structure.

Task:

- Build an endpoint `/api/ask` that:

Accepts a POST request:

```
{
  "question": "What is LangChain?"
}
```

    -
    - Calls any **cloud-hosted LLM API**, such as:
        - **OpenAI API** (`gpt-3.5-turbo` or higher)
        - **Hugging Face Inference API**
    - Returns the model's response as JSON.

Expected Output Example:

```
{
  "answer": "LangChain is a framework for building applications powered by large language models."
}
```

Bonus:

- Add simple caching or logging (prompt, timestamp, model used).

- Add error handling for API key or rate-limit issues.

Test Case 2 – Vector Database Integration

Objective: Assess understanding of embeddings and vector retrieval.

Task:

1. Choose a **Vector DB** (Chroma, FAISS, Pinecone, or Weaviate).
2. Insert at least 5 short text documents (paragraphs or articles).
3. Build `/api/search` endpoint that:

Accepts a POST request:

{"query": "What is machine learning?"}

-
- Uses an embedding model (e.g., `text-embedding-3-small`, `sentence-transformers/all-MiniLM-L6-v2`)
- Performs a similarity search against your stored documents
- Returns the top 3 relevant matches

Expected Output Example:

```
{
  "query": "What is machine learning?",
  "results": [
    {"text": "Machine learning is a subset of artificial intelligence...", "score": 0.92},
    {"text": "Supervised learning uses labeled data...", "score": 0.88}
  ]
}
```

Bonus:

- Add metadata fields (title, source).
- Visualize similarity scores in the response.

Test Case 3 – Simple RAG Backend

Objective: Evaluate ability to combine LLM and Vector DB for contextual generation.

Task:

- Build `/api/rag` endpoint that:
    - Accepts a user query (same as previous task).
    - Searches your Vector DB for the top relevant texts.
    - Constructs a **contextual prompt** using the retrieved documents.
    - Sends it to your LLM API and returns a contextual answer.

Example Prompt Template:

Use the following context to answer the question:

Context:
<top 3 retrieved documents>

Question:
<user query>

Answer based on the context:

Expected Output Example:

```
{
  "query": "Who developed LangChain?",
  "context_used": ["LangChain is developed by Harrison Chase...", "..."],
  "final_answer": "LangChain was developed by Harrison Chase."
}
```

Test Case 4 – Concept: Graph RAG

Objective: Evaluate conceptual understanding (no coding required).

Task:
Write a short explanation (max 200 words):

- What is **Graph RAG**?
- How would **Neo4J** help improve information retrieval and reasoning?
- Example of how entities and relationships can enhance context for LLMs.